

EGYPTIAN RUSSIAN UNIVERSITY (ERU)

Faculty of Artificial Intelligence
AI Applications Course — Spring 2026

AGRIVISION AI

End-to-End Vision-RAG Decision Support System for Precision Crop Disease Diagnosis

Technical Project Documentation

Project Team

Ashraf Ghoniem 225115 | Akram Mohamed 225186 | Youssef Lasheen 225222
Mostafa Mohamed 225268 | Abdelrahman Ahmed 225277

Supervisor: Dr. Mohammed Al-Gunaid, Ph.D. | Associate Professor

May 2026

Repository: github.com/youseflasheen/Vision-RAG-AgriSystem

Abstract

Modern agricultural AI systems face a critical reliability gap: standard Large Language Models (LLMs) frequently hallucinate incorrect pesticide dosages and treatment protocols when queried about crop diseases, potentially causing catastrophic crop loss. Simultaneously, conventional Convolutional Neural Networks (CNNs) classify diseases but leave farmers without actionable intervention plans.

This document presents AgriVision AI — an industrial-grade, microservice-based decision-support system that bridges this gap through a novel Vision-RAG (Retrieval-Augmented Generation) architecture. The system integrates five core AI modules into a unified pipeline:

1. A fine-tuned ResNet-50 vision model achieving 90.9% validation accuracy on a 21-class hybrid dataset (28,859 images from PlantVillage and PlantDoc).
2. A DCGAN generator producing 1,400 synthetic images to balance underrepresented disease classes.
3. A ChromaDB vector database encoding 21 verified agricultural protocols using SentenceTransformer embeddings.
4. A Llama 3.3 70B LLM accessed via Groq LPU generating context-grounded treatment reports.
5. An SIR epidemiological simulation engine modelling pest spread under three intervention strategies (No Treatment, Pesticide Application, Quarantine).

A weighted multi-criteria Decision Support System (DSS) ranks interventions by combining crop loss projections (60%), model confidence (25%), and RAG relevance (15%). The system is deployed via FastAPI and Streamlit with Docker containerization, providing farmers a complete end-to-end workflow — from leaf image upload to ranked treatment recommendations — in under 120 seconds.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1. Introduction	5
1.1 Background and Problem Statement.....	5
1.2 Motivation	5
1.3 Objectives.....	6
1.4 Scope	6
2. Related Work	7
2.1 Plant Disease Detection Systems	7
2.2 RAG in Agriculture.....	7
2.3 Comparison with Existing Systems	7
2.4 Key Differentiators	8
3. System Overview	9
3.1 Architecture	9
3.2 Workflow.....	9
3.3 Module Descriptions	11
4. Dataset and Data Collection	12
4.1 Dataset Sources	12
4.2 Preprocessing Pipeline.....	12
4.3 Dataset Statistics.....	12
4.4 Training Data Augmentation.....	13
4.5 Knowledge Base	13
4.6 Data Challenges.....	13
5. Technologies Used	14
5.1 Technology Stack.....	14
5.2 Hardware Requirements.....	14
6. Implementation	15
6.1 Project Structure.....	15
6.2 Vision Model.....	15
6.3 RAG Pipeline.....	16
6.4 Prompt Engineering.....	16
6.5 GAN Implementation	17
6.6 Simulation (SIR Model).....	17
6.7 Decision Support System (DSS)	18
6.8 Backend API.....	18

6.9 Frontend Dashboard	19
7. System Diagrams.....	20
7.1 Use Case Diagram	20
7.2 Activity Diagram	21
7.3 Sequence Diagram.....	22
7.4 Data Flow Diagram (DFD)	23
8. Results and Testing	24
8.1 Vision Model Performance	24
8.2 GAN Training Results.....	24
8.3 End-to-End Test Cases	24
8.4 System Limitations	25
9. UI/UX and Screens	26
9.1 Dashboard Overview	26
9.2 Analysis Tab — Image Upload	26
9.3 Disease Detection Result Card	27
9.4 Chatbot Interface.....	27
9.5 Simulation Visualisation.....	28
9.6 DSS Rankings Table	28
10. Challenges and Solutions	29
11. Future Work	30
11.1 Model Improvements.....	30
11.2 System Expansion.....	30
11.3 Advanced Features	30
11.4 Research Directions	30
12. Conclusion	31
Key Achievements.....	31
Lessons Learned.....	31
13. References.....	32
Academic References	32
Framework and Tool Documentation	32
Dataset References.....	32
Project Repository	32

1. Introduction

1.1 Background and Problem Statement

Global agriculture faces an unprecedented challenge: by 2050, the world population is projected to reach 9.7 billion, requiring a 60% increase in food production while arable land shrinks and water resources dwindle. Plant diseases alone account for 30–40% of annual crop yield losses worldwide. Early detection and precise treatment are the most effective countermeasures, yet traditional diagnostic methods — manual visual inspection by agricultural experts — are slow, subjective, and inaccessible to smallholder farmers who constitute 84% of the world's farms.

The emergence of AI-powered diagnostic tools offers transformative potential. However, two fundamental technical problems persist in current systems:

1. The Hallucination Problem: General-purpose LLMs queried about agricultural diseases frequently generate plausible-sounding but chemically incorrect treatment recommendations — wrong fungicide concentrations, non-existent products, or inappropriate application methods. In high-stakes agriculture, such misinformation can destroy entire harvests.

2. The Action Gap: Even accurate CNN-based classifiers leave the farmer without a concrete treatment plan. Classification without actionable decision support is diagnostically complete but operationally incomplete.

1.2 Motivation

This project is motivated by the real-world water and agricultural infrastructure challenges at the Egyptian Russian University (ERU) campus, where fragmented monitoring systems, sensor blind zones, and lack of integrated decision-making tools mirror the broader agricultural sector's data-integration deficit. The ERU case study demonstrates that the core barrier to intelligent resource management is not infrastructure scarcity but a lack of integration and intelligence.

AgriVision AI addresses this by demonstrating how multiple AI paradigms — computer vision, vector retrieval, LLM generation, generative adversarial networks, and epidemiological simulation — can be orchestrated into a single coherent system that is simultaneously accurate, actionable, and accessible.

1.3 Objectives

The project has five primary objectives:

1. Design and train a high-accuracy vision model (ResNet-50) on a hybrid laboratory-field dataset spanning 21 crop disease classes.
2. Implement a Retrieval-Augmented Generation (RAG) pipeline that eliminates hallucination by grounding LLM responses in a verified ChromaDB knowledge base.
3. Develop an SIR epidemiological simulation modelling disease spread under competing intervention strategies.
4. Build a multi-criteria DSS that algorithmically ranks interventions using weighted scoring.
5. Deploy the complete system as a connected FastAPI + Streamlit application demonstrating full end-to-end workflow.

1.4 Scope

The system covers 21 crop-disease classes across six crop species: Apple, Corn, Potato, Tomato, Cherry, Grape, Peach, Pepper, and Strawberry. Input is a single RGB leaf image; output is a structured DSS report containing disease identification, confidence score, expert-grounded treatment protocol, simulation projections, and ranked intervention strategies. The system is designed for demonstration and educational purposes with a clear pathway to production deployment.

2. Related Work

2.1 Plant Disease Detection Systems

Several systems in the literature address plant disease detection using deep learning. Mohanty et al. (2016) demonstrated that CNNs trained on the PlantVillage dataset could achieve over 99% accuracy under laboratory conditions, but performance degraded significantly on field-captured images due to lighting variations and complex backgrounds. The PlantDoc dataset (Singh et al., 2020) was specifically created to address this domain shift by collecting images from actual farm environments.

2.2 RAG in Agriculture

Retrieval-Augmented Generation (Lewis et al., 2020) has been applied to medical and legal domains to ground LLM outputs, but agricultural applications remain nascent. The Agricultural Extension Agent (Rhode01, 2025) demonstrated a ReAct agent architecture for maize-leaf diagnosis with knowledge retrieval from agricultural guides. AgriPal (Avidan87, 2025) introduced multi-agent orchestration for crop analysis but relied on API-based vision services rather than custom-trained models.

2.3 Comparison with Existing Systems

Feature	Mohanty et al.	AgriPal	Ag. Extension Agent	AgriVision AI (This Work)
Vision Model	AlexNet/GoogLeNet	API-based	Custom CNN	Fine-tuned ResNet-50
Dataset	Lab-only (PlantVillage)	Not specified	Maize only	Hybrid PlantVillage + PlantDoc (21 classes)
Hallucination Prevention	N/A	Not addressed	Partial (RAG)	Full RAG + ChromaDB persistence
Synthetic Data	No	No	No	DCGAN augmentation (1,400 images)
Disease Simulation	No	No	No	SIR epidemiological model
Decision Support	No	Partial	Yes	Weighted multi-criteria DSS
Deployment	Research	Cloud API	Prototype	Docker + FastAPI + Streamlit

Table 2.1: Comparison of AgriVision AI with existing plant disease detection systems.

2.4 Key Differentiators

AgriVision AI distinguishes itself through three architectural innovations:

1. **Vision-RAG Fusion:** Unlike systems that treat vision classification and LLM generation as separate tools, AgriVision couples them tightly — the predicted disease class triggers a targeted semantic search in ChromaDB, and the retrieved context becomes the mandatory grounding for LLM generation.
2. **Simulation-Augmented DSS:** Rather than simply recommending a treatment, the system simulates disease progression under multiple intervention scenarios, enabling quantitative comparison.
3. **GAN-Based Class Balancing:** The DCGAN generates photorealistic synthetic samples for underrepresented classes, addressing the well-known long-tail distribution problem in agricultural datasets.

3. System Overview

3.1 Architecture

AgriVision AI follows a layered microservice architecture with nine integrated modules deployed across two runtime processes: a FastAPI backend server and a Streamlit frontend client. All AI processing engines are encapsulated as independent, independently testable modules orchestrated by the FastAPI layer.

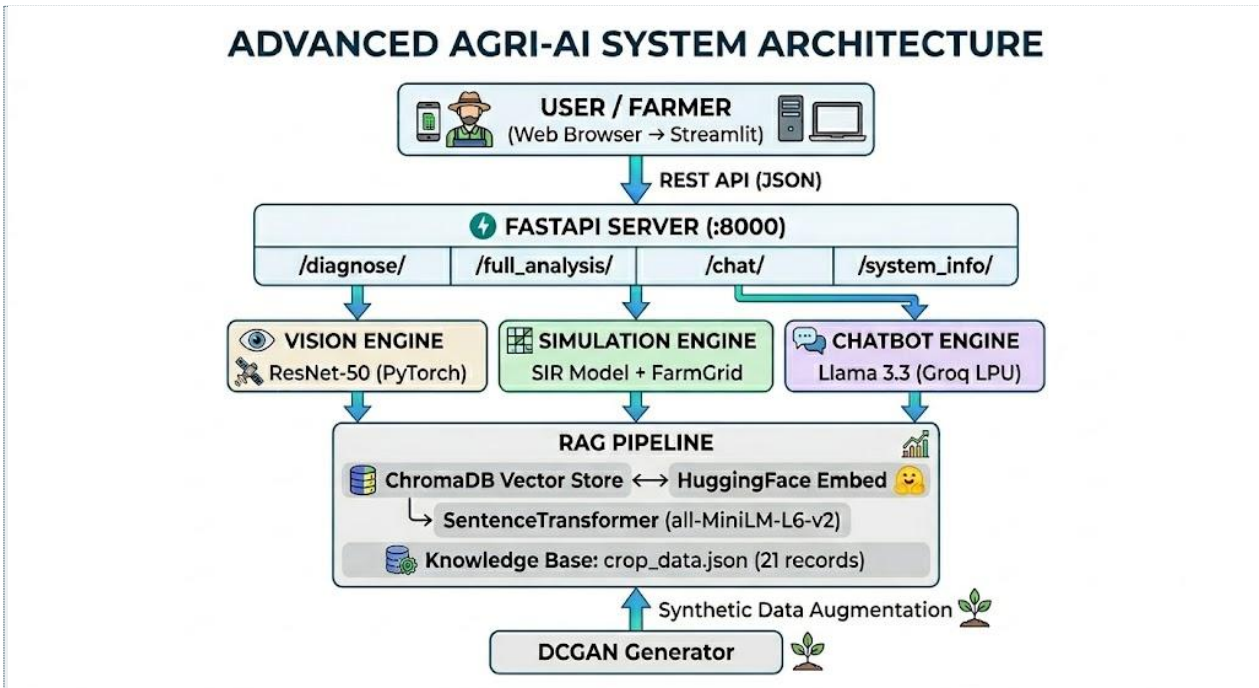


Figure 3.1: System Architecture showing the interaction between user-facing UI, FastAPI orchestration layer, and all AI processing engines.

3.2 Workflow

The complete user workflow proceeds through seven sequential stages:

1. **Image Upload:** The farmer uploads a leaf image (JPG/PNG) through the Streamlit dashboard.
2. **Vision Inference:** The image is preprocessed (resized to 256×256, center-cropped to 224×224, normalized to ImageNet statistics) and passed through the fine-tuned ResNet-50 model, producing a 21-class probability distribution.
3. **RAG Retrieval:** The predicted disease class triggers a semantic search in ChromaDB. The vector store returns the top-1 matching document containing verified symptoms, treatment, and prevention information.
4. **LLM Generation:** The retrieved context is combined with a structured prompt template and sent to Llama 3.3 70B via Groq API. The LLM generates a Markdown-formatted treatment protocol strictly grounded in the retrieved context.

5. Simulation: The SIR model runs three scenarios (No Intervention, Pesticide, Quarantine) on a 20×20 farm grid over 30 simulated days using disease-specific transmission rates (β).
6. DSS Scoring: Each intervention scenario receives a composite score weighted by projected crop loss (60%), model confidence (25%), and RAG retrieval relevance (15%).
7. Report Generation: All outputs are assembled into a structured JSON report and returned to the Streamlit dashboard for visualization.

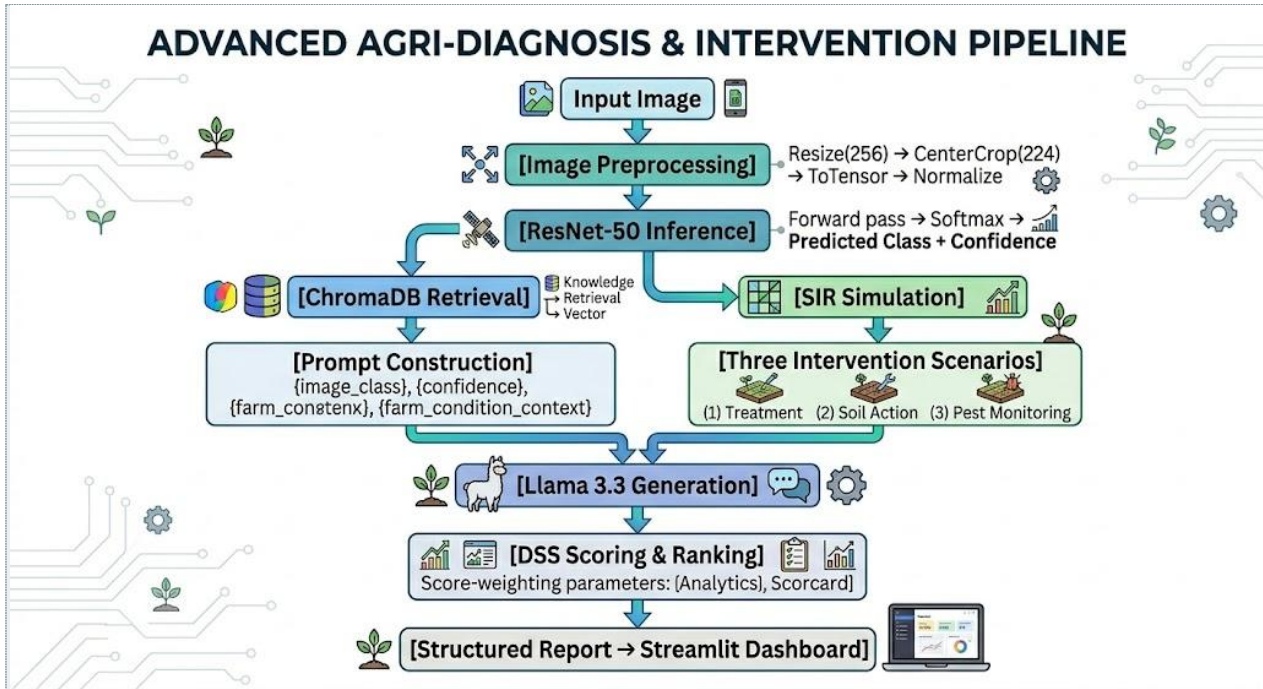


Figure 3.2: AI Pipeline Flow showing data transformations at each processing stage.

3.3 Module Descriptions

Module	Purpose	Technology	Input	Output	Lab
Chatbot	Domain Q&A for farmers	Groq Llama 3.3 70B	NL questions	Context-grounded answers	Lab 1
Prompts	Centralized prompt templates	Pure Python	Disease class, context	Formatted LLM prompts	Lab 2
Data	Data collection & preprocessing	PyTorch DataLoader	Raw images	Normalized tensors	Lab 3
RAG	Vector retrieval pipeline	ChromaDB + HuggingFace	Disease class query	Retrieved knowledge passages	Lab 4
Model	Vision classification	ResNet-50 (PyTorch)	Leaf image tensor	Predicted class + confidence	Lab 5
GAN	Synthetic data generation	DCGAN (PyTorch)	Noise + class labels	64×64 synthetic leaf images	Lab 6
Simulation	Pest spread modelling	Custom SIR + FarmGrid	Disease name, grid params	Daily infection counts, peak infected	Lab 7
Dashboard	Visualization UI	Streamlit, Plotly	DSS report JSON	Interactive charts, metrics	Lab 8
DSS	Decision scoring & ranking	Weighted multi-criteria scoring	Simulation results, confidence	Ranked intervention strategies	Lab 9

Table 3.1: System module descriptions with technologies, inputs, and outputs.

4. Dataset and Data Collection

4.1 Dataset Sources

The system uses a hybrid dataset constructed from two primary sources:

PlantVillage Dataset: 27,335 laboratory-condition leaf images across 38 disease classes. Images are captured against uniform backgrounds with controlled lighting.

PlantDoc Dataset: 1,524 field-condition images across 28 classes. Images are captured in natural farm environments with variable lighting, complex backgrounds, and multiple leaves per frame.

4.2 Preprocessing Pipeline

The hybrid dataset construction follows a rigorous four-step preprocessing pipeline:

1. **Class Selection:** From the 38 PlantVillage and 28 PlantDoc classes, 21 target classes are selected covering six major crop species — Apple (4 classes), Corn/Maize (4 classes), Potato (3 classes), and Tomato (10 classes).
2. **Cross-Dataset Mapping:** PlantDoc folder names are mapped to PlantVillage standard class names using a curated mapping table (e.g., 'Tomato Early blight leaf' → 'Tomato__Early_blight').
3. **File Renaming and Merging:** PlantVillage images are prefixed with `pv_` and PlantDoc images with `wild_train_` or `wild_test_` to preserve provenance information while enabling unified training.
4. **GAN Augmentation:** For classes with fewer than 1,000 images, a Conditional DCGAN generates 200 synthetic images per class, producing 1,400 additional training samples across seven underrepresented classes.

4.3 Dataset Statistics

Split	Images	Percentage
Training	24,207	80%
Validation	3,025	10%
Test	3,027	10%
Total	30,259	100%

Table 4.1: Dataset split statistics after GAN augmentation.

Crop	Classes	Total Images (After Augmentation)
Apple	4	3,444

Corn/Maize	4	4,228
Potato	3	2,574
Tomato	10	20,013

Table 4.2: Class distribution by crop species.

4.4 Training Data Augmentation

During ResNet-50 training, the following stochastic augmentations are applied to improve generalization across lab and field conditions:

- Random horizontal flip ($p = 0.5$)
- Random vertical flip ($p = 0.2$)
- Random rotation ($\pm 30^\circ$)
- Color jitter (brightness = 0.3, contrast = 0.3, saturation = 0.3, hue = 0.1)

4.5 Knowledge Base

The RAG pipeline's knowledge base (data/knowledge_base/crop_data.json) contains 21 structured records, each with fields: class_name, crop, disease, symptoms, treatment, and prevention. These records are sourced from agricultural extension databases and verified against published protocols.

4.6 Data Challenges

Challenge	Description	Solution
Domain Shift	Lab (PlantVillage) vs. field (PlantDoc) imaging conditions differ significantly.	Hybrid dataset construction merges both sources; aggressive augmentation during training.
Class Imbalance	Tomato Yellow Leaf Curl Virus: 5,433 images vs. Potato Healthy: 152 images.	DCGAN generates 200 synthetic images per underrepresented class.
Mapping Ambiguity	Some PlantDoc classes lack direct PlantVillage equivalents.	Manual curated mapping table constructed for cross-dataset alignment.

Table 4.3: Data challenges and implemented solutions.

5. Technologies Used

5.1 Technology Stack

Category	Technology	Version	Purpose
Language	Python	3.12	All implementation
Deep Learning	PyTorch	2.10.0	ResNet-50 model, DCGAN training
	torchvision	Latest	Image transforms, pre-trained models
	CUDA	12.8	GPU acceleration (Tesla T4)
NLP / Embeddings	Sentence-Transformers	Latest	all-MiniLM-L6-v2 embeddings
	LangChain	Latest	RAG chain orchestration
	LangChain-Groq	Latest	Groq LLM integration
Vector Database	ChromaDB	Latest	Persistent vector storage
LLM Inference	Groq API (Llama 3.3 70B)	Latest	Treatment protocol generation
Backend	FastAPI	Latest	REST API server
	Uvicorn	Latest	ASGI server
Frontend	Streamlit	Latest	Interactive dashboard
	Plotly	Latest	Interactive charts
DevOps	Docker / Docker Compose	Latest	Containerization + multi-service orchestration
Development	Google Colab	—	Model training on T4 GPU

Table 5.1: Full technology stack.

5.2 Hardware Requirements

Component	Training	Inference
GPU	NVIDIA T4 (16 GB VRAM)	CPU or GPU
RAM	16 GB	8 GB
Storage	5 GB	2 GB
Network	Required (dataset download, Groq API)	Required (Groq API)

Table 5.2: Hardware requirements for training and inference.

6. Implementation

6.1 Project Structure

The repository follows a modular lab-based structure where each lab directory corresponds to a distinct system component:

```

Vision-RAG-AgriSystem/
├── api/main.py                # FastAPI application (4 endpoints)
├── app/frontend.py           # Streamlit dashboard (6 tabs)
├── data/
│   ├── knowledge_base/crop_data.json  # 21 verified disease records
│   ├── processed/                  # Hybrid dataset
│   └── synthetic/                  # GAN-generated images
├── models/
│   ├── resnet50_hybrid_best.pth      # Trained ResNet-50 weights
│   ├── class_mapping.json           # Class index → name mapping
│   └── gan_generator_v1.pth         # Trained DCGAN generator
├── src/
│   ├── lab1_chatbot/chatbot.py      # AgriculturalChatbot class
│   ├── lab2_prompts/prompt_templates.py # Centralized prompt library
│   ├── lab4_rag/                    # RAGGenerator +
│   ├── AgriculturalVectorDB
│   ├── lab5_model/inference.py      # AgriVisionRAG class
│   ├── lab6_gan/                   # DCGAN model definitions
│   ├── lab7_simulation/             # SIR + FarmGrid +
│   ├── interventions
│   │   └── lab9_dss/                # DSS engine + scoring + report
│   ├── vector_db/                  # ChromaDB persistence
│   ├── Dockerfile
│   └── requirements.txt

```

6.2 Vision Model

The vision classification engine is built on ResNet-50 pre-trained on ImageNet1K_V2 weights, with a custom classification head replacing the original fully connected layer:

```

fc = Sequential(
    Linear(2048, 512),
    ReLU(),
    BatchNorm1d(512),
    Dropout(0.5),
    Linear(512, 21)    # 21 crop-disease classes
)

```

Training configuration:

- Optimizer: AdamW (lr = 0.001, weight_decay = 0.01)
- Loss: CrossEntropyLoss with label smoothing ($\epsilon = 0.1$)
- Scheduler: ReduceLROnPlateau (factor = 0.1, patience = 3)
- Batch size: 32 | Input size: 224×224×3 | Epochs: 15 (early stopping, patience = 5)
- Best validation accuracy: 90.91% at Epoch 14 | Training time: ~55 min on NVIDIA T4

6.3 RAG Pipeline

The RAG pipeline consists of three tightly integrated components:

Vector Store (AgriculturalVectorDB):

- Embedding model: sentence-transformers/all-MiniLM-L6-v2 (384-dimensional vectors)
- Database: ChromaDB with persistent storage at vector_db/
- Retrieval: Top-1 semantic search using cosine similarity

RAG Generator (RAGGenerator):

- LLM: llama-3.3-70b-versatile via Groq API, temperature = 0.3
- Prompt: Structured role as 'Senior Agricultural Engineer' with mandatory four-section output — Disease Identification, Key Symptoms, Treatment Plan, Prevention Strategy
- Chain: {context: retriever, disease_class: RunnablePassthrough()} → prompt_template → llm → StrOutputParser()

6.4 Prompt Engineering

All prompts are centralized in src/lab2_prompts/prompt_templates.py. Three persona definitions and three template builders are defined:

1. AGRICULTURAL_ENGINEER_PERSONA: Used by the RAG generator. Specifies 20 years of field experience, specialization in integrated pest management, and strict grounding rules.
2. CHATBOT_PERSONA: Used by the farmer-facing chatbot. Emphasizes accessibility ('plain language a farmer can act on immediately').
3. build_disease_analysis_prompt(): Constructs the structured RAG prompt with four mandatory output sections.
4. build_chatbot_system_prompt(): Injects RAG-retrieved context into the chatbot's system message for session-level grounding.
5. build_followup_prompt(): Handles multi-turn conversation by appending follow-up context to existing history.

6.5 GAN Implementation

A Conditional DCGAN addresses class imbalance by generating synthetic leaf images:

Generator Architecture:

- Input: Random noise (100-dim) + class embedding (50-dim)
- Four transposed convolution blocks: $512 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 3$ channels
- Output: $64 \times 64 \times 3$ RGB image with Tanh activation

Discriminator Architecture:

- Input: $64 \times 64 \times 4$ (3 image channels + 1 label embedding channel)
- Four convolution blocks outputting a 4×4 real/fake classification with Sigmoid activation

Training:

- Adam ($\text{lr} = 0.0002$, $\beta_1 = 0.5$, $\beta_2 = 0.999$), BCE Loss, batch size 64, 30 epochs
- Seven underrepresented classes received 200 synthetic images each (1,400 total)

6.6 Simulation (SIR Model)

The pest spread simulation implements a spatially explicit SIR (Susceptible-Infected-Removed) model over a 20×20 FarmGrid:

- Three initial infections are seeded at the grid centre on Day 0.
- At each daily time step, infected cells attempt to infect each of 8 Moore-neighbourhood susceptible neighbours with probability β .
- Infected cells transition to REMOVED with probability γ (recovery rate).

Disease-specific transmission rates:

Disease	β (Transmission Rate)
Potato Late Blight	0.50
Tomato Late Blight	0.45
Corn Northern Leaf Blight	0.35
Corn Common Rust	0.32
Tomato Early Blight	0.30
Tomato Septoria Leaf Spot	0.28
Default	0.25

Table 6.1: Disease-specific SIR transmission rates (β).

Three intervention strategies are evaluated independently:

1. No Intervention (Baseline): Models natural disease progression.
2. Pesticide Application: Reduces β to 30% of its original value, simulating fungicide protection.
3. Quarantine: Removes all cells within Manhattan distance 2 from each infected cell, creating a physical firebreak.

6.7 Decision Support System (DSS)

The DSS orchestrates the complete pipeline and applies the following composite scoring formula:

$$\text{final_score} = (\text{crop_loss_normalized} \times 0.60) - (\text{model_confidence} \times 0.25) - (\text{rag_relevance} \times 0.15)$$

Lower scores indicate better interventions. Crop loss drives 60% of the decision because minimising yield loss is the primary agricultural objective. Urgency is classified as HIGH (confidence $\geq 80\%$ AND peak infection $\geq 40\%$), MEDIUM (either condition), or LOW (neither condition).

6.8 Backend API

Endpoint	Method	Input	Output	Resp. Time
/	GET	—	Health check	<10 ms
/diagnose/	POST	Image file	Disease class + confidence + RAG advice	~8 s
/full_analysis/	POST	Image file	Complete DSS report	~15 s
/chat/	POST	JSON {"question": "..."} {"..."} {"..."}	Chatbot answer	~4 s
/system_info/	GET	—	Component status	<10 ms

Table 6.2: FastAPI endpoints with inputs, outputs, and typical response times.

All heavy models (ResNet-50, ChromaDB, Chatbot) are loaded once at server startup using FastAPI's `@asyncontextmanager` lifespan hook, avoiding per-request cold-start overhead.

6.9 Frontend Dashboard

The Streamlit dashboard employs glassmorphism design patterns, a deep space gradient background, and custom CSS injection (Inter + JetBrains Mono fonts). Six interactive tabs are provided:

1. Analyse: Image upload, pipeline execution, urgency-coded result cards, and RAG protocol display.
2. Chatbot: Session-based multi-turn farmer Q&A with context retention (max 6 turns).
3. Simulation: Plotly line chart overlaying infection trajectories for all three intervention scenarios.
4. DSS: Ranked intervention table with crop loss percentage and heuristic scores.
5. GAN Data: Educational content on synthetic data augmentation.
6. About: Real-time system component status from the `/system_info/` endpoint.

7.2 Activity Diagram

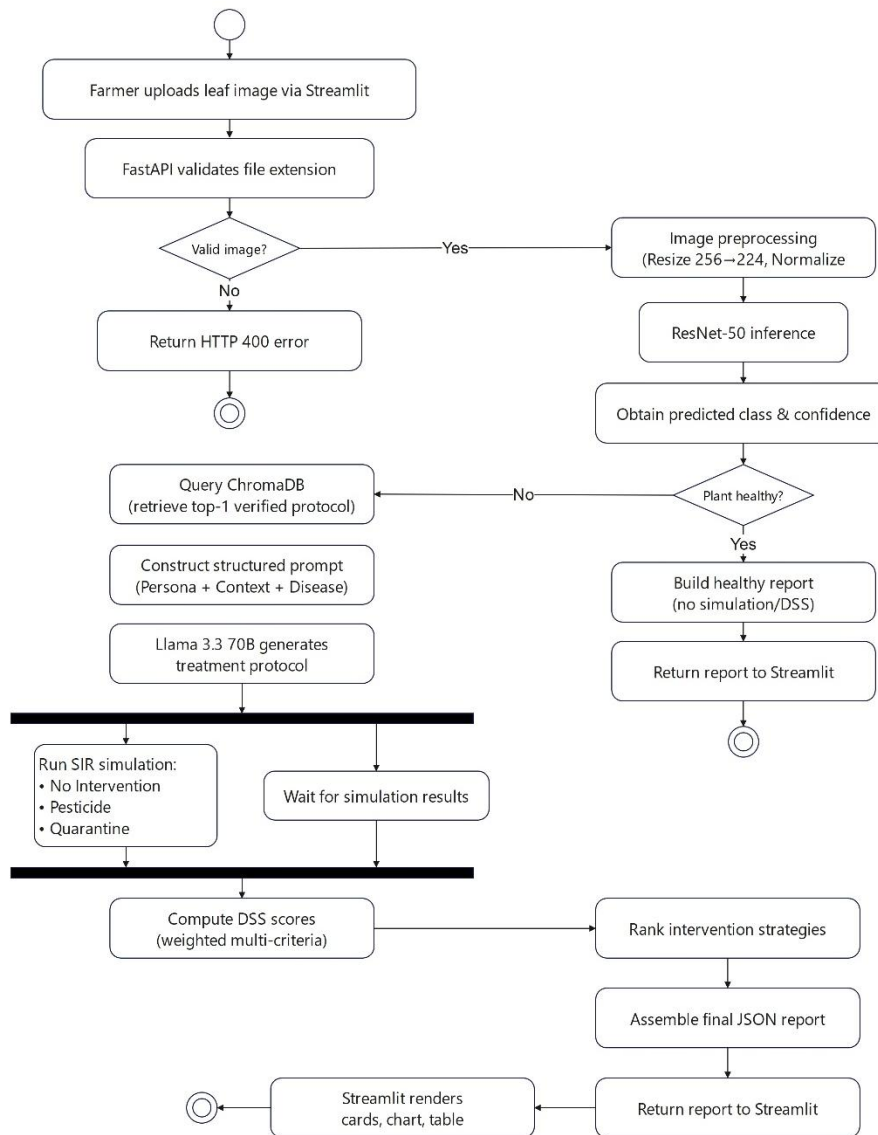


Figure 7.2: Activity Diagram showing complete system workflow from image upload to report rendering, including healthy plant short-circuit path.

7.3 Sequence Diagram

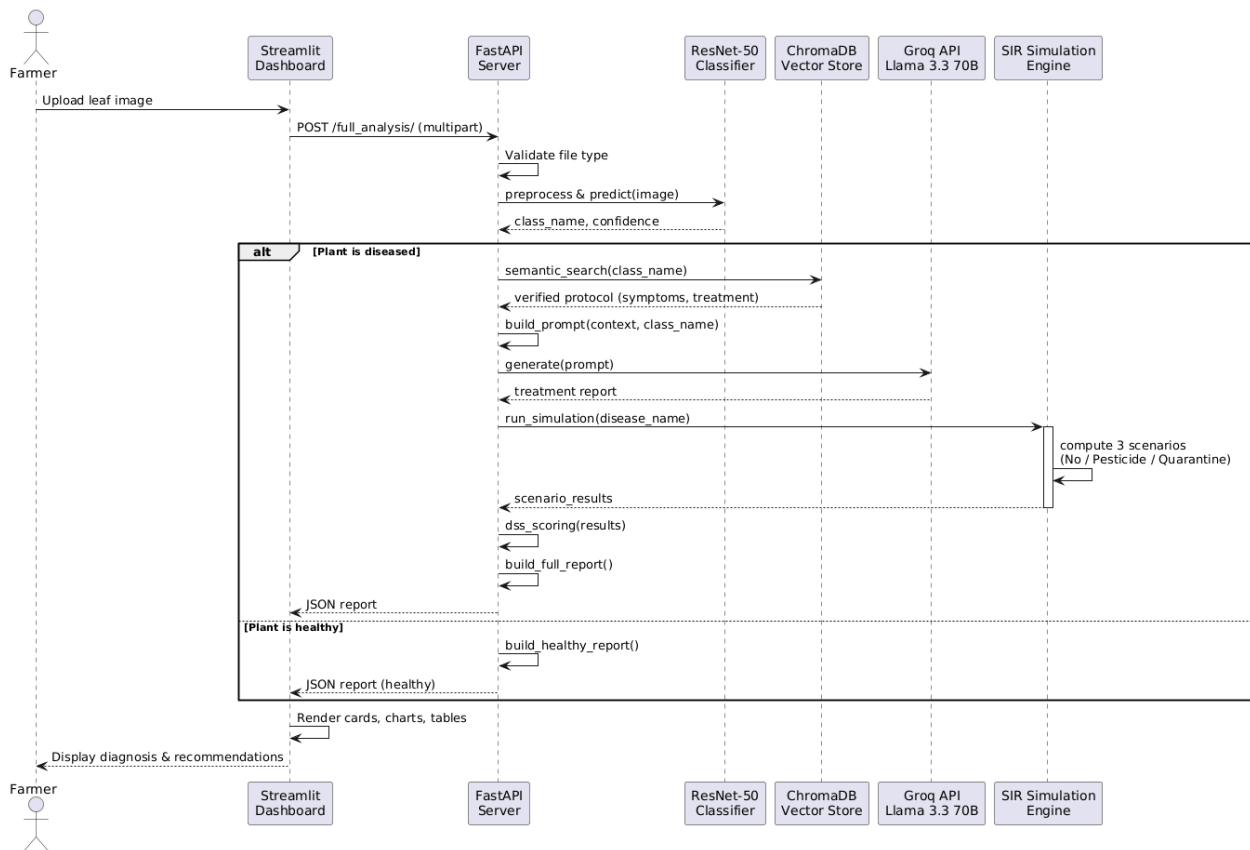


Figure 7.3: Sequence Diagram illustrating temporal interactions between Farmer, Streamlit, FastAPI, ResNet-50, ChromaDB, Groq LLM, and Simulation Engine.

7.4 Use Case Diagram

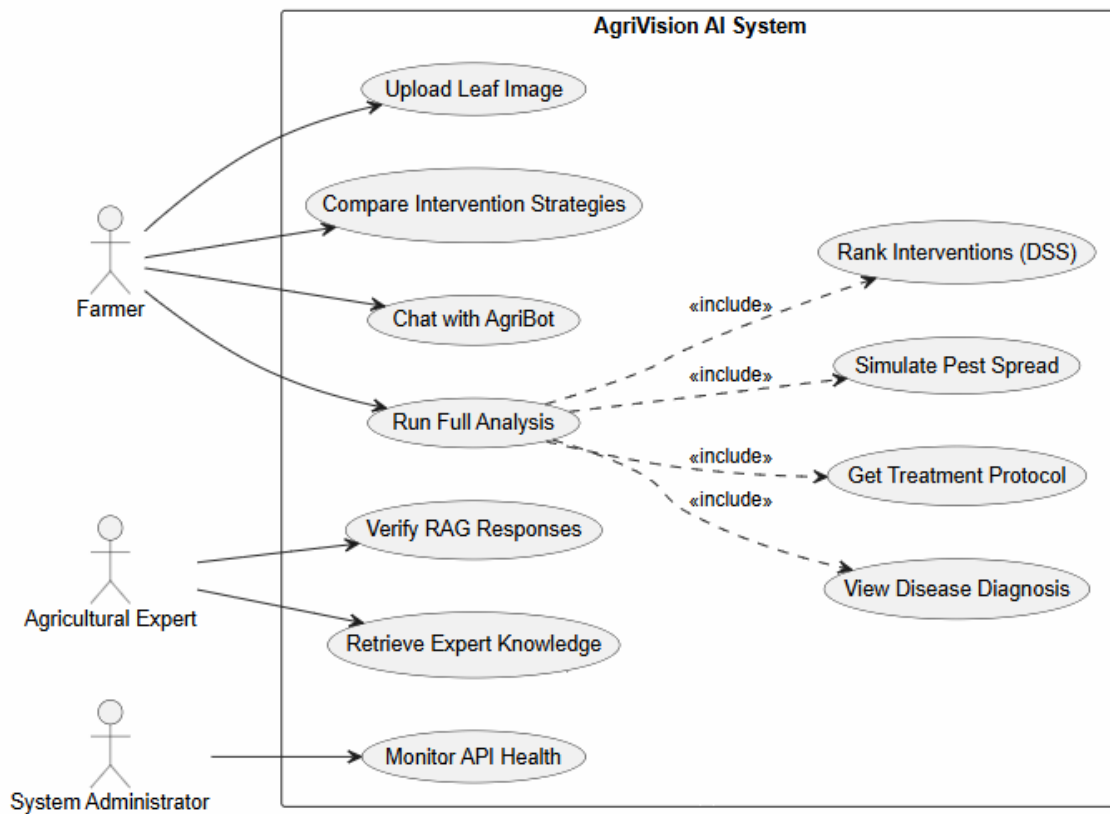


Figure 7.4: Use Case Diagram. Primary actor (Farmer) follows the workflow: upload → diagnose → treat → compare → ask. Secondary actors: Agricultural Expert (validates RAG responses), System Administrator (monitors API health).

8. Results and Testing

8.1 Vision Model Performance

The ResNet-50 model was trained for 15 epochs with early stopping (patience = 5). Key training metrics are summarised below:

Epoch	Train Loss	Train Acc	Val Loss	Val Acc	Note
1	1.3038	75.59%	1.0750	83.90%	
4	1.1025	83.38%	0.9836	87.37%	
8	1.0498	85.59%	0.9336	90.71%	
13	1.0041	87.26%	0.9208	90.18%	
14	0.9865	87.83%	0.9063	90.91%	★ Best
15	0.9764	88.29%	0.9237	90.05%	

Table 8.1: ResNet-50 training history. Best model saved at Epoch 14 (90.91% validation accuracy).

The training curves demonstrate healthy convergence with no significant overfitting gap, attributable to aggressive data augmentation and dropout regularisation.

8.2 GAN Training Results

The DCGAN converged over 30 epochs, with generator loss stabilising around 7.36 and discriminator loss around 0.18. Generated synthetic images at epoch 30 exhibited recognisable leaf structures and disease patterns suitable for data augmentation.

8.3 End-to-End Test Cases

Test Case	Vision Output	Simulation	DSS Recommendation
Case 1: Tomato Septoria Leaf Spot	Tomato___Septoria_leaf_spot (94% confidence)	Pesticide: 28% crop loss vs. 62% baseline	Pesticide Application (Rank 1)
Case 2: Corn Northern Leaf Blight	Corn_(maize)___Northern_Leaf_Blight (high confidence)	Quarantine shows best containment across three scenarios	Quarantine recommended with expert fungicide protocol

Table 8.2: End-to-end test case results.

8.4 System Limitations

- **Single-Image Analysis:** The system processes one leaf at a time; multi-leaf or whole-plant analysis is not yet supported.
- **Internet Dependency:** The Groq API requirement prevents fully offline operation.
- **Disease Scope:** Limited to 21 classes; rare or regional diseases outside this set cannot be diagnosed.
- **Simulation Simplification:** The SIR model uses a uniform grid; real farms have heterogeneous terrain, microclimates, and mixed-crop planting.

9. UI/UX and Screens

9.1 Dashboard Overview

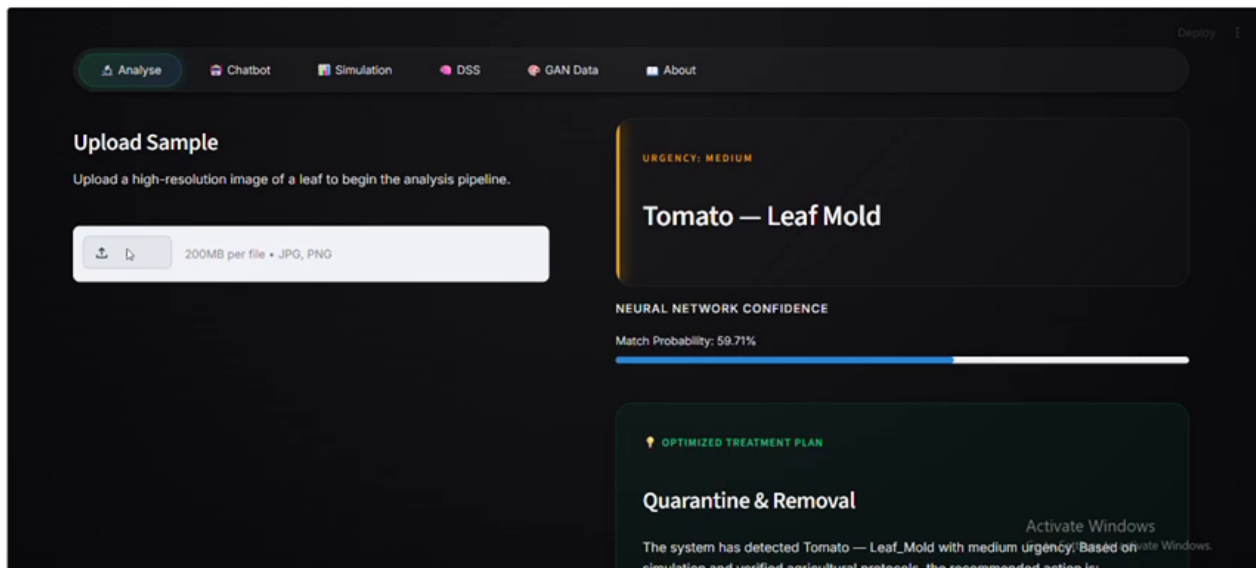


Figure 9.1: Main dashboard header with 'System Online' health indicator. The pulsing green dot signals that the FastAPI backend is operational.

9.2 Analysis Tab — Image Upload

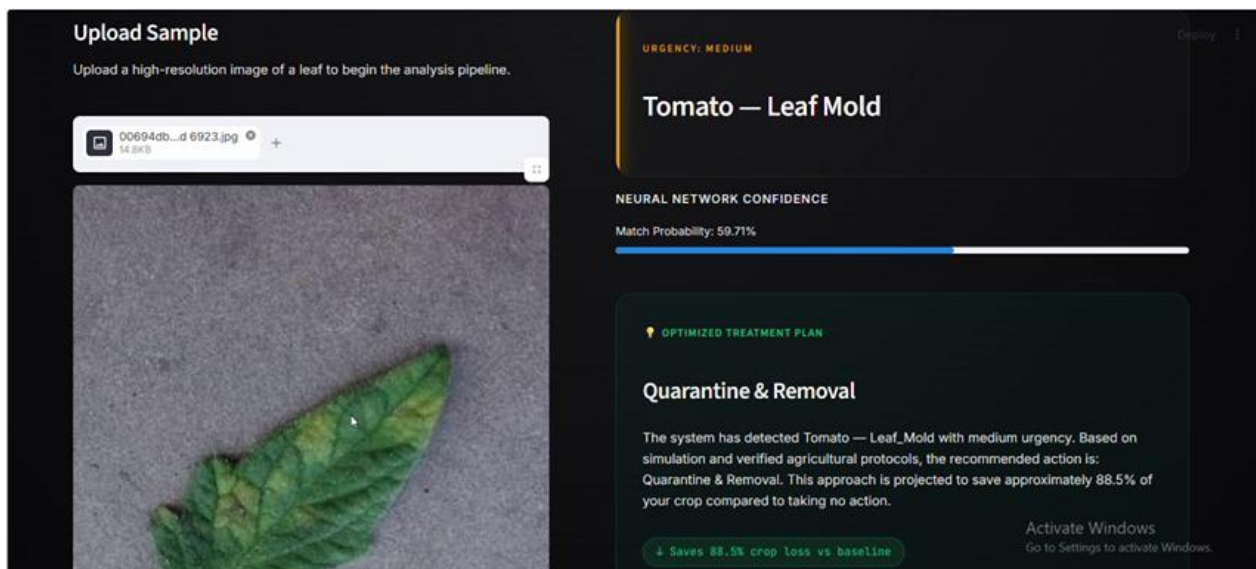


Figure 9.2: Analysis tab with drag-and-drop image upload (left) and 'Initialize Deep Scan' button (right). Triggers the complete Vision → RAG → Simulation → DSS pipeline.

9.3 Disease Detection Result Card

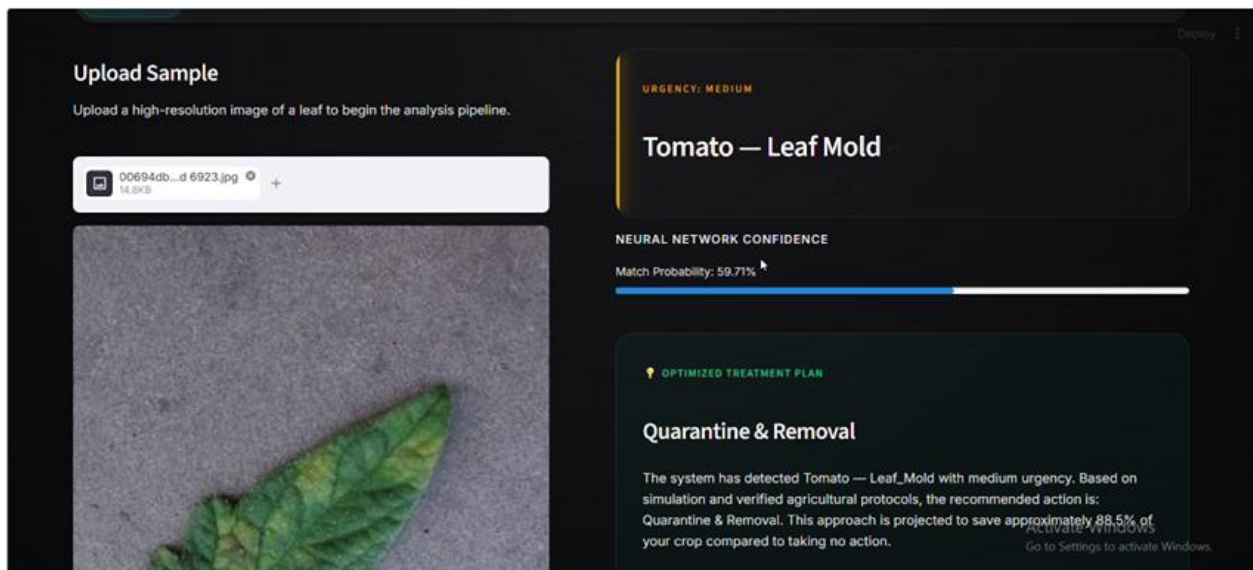


Figure 9.3: Disease detection result card showing urgency badge (HIGH/MEDIUM/LOW), disease name, model confidence bar, and optimised treatment plan with savings percentage.

9.4 Chatbot Interface

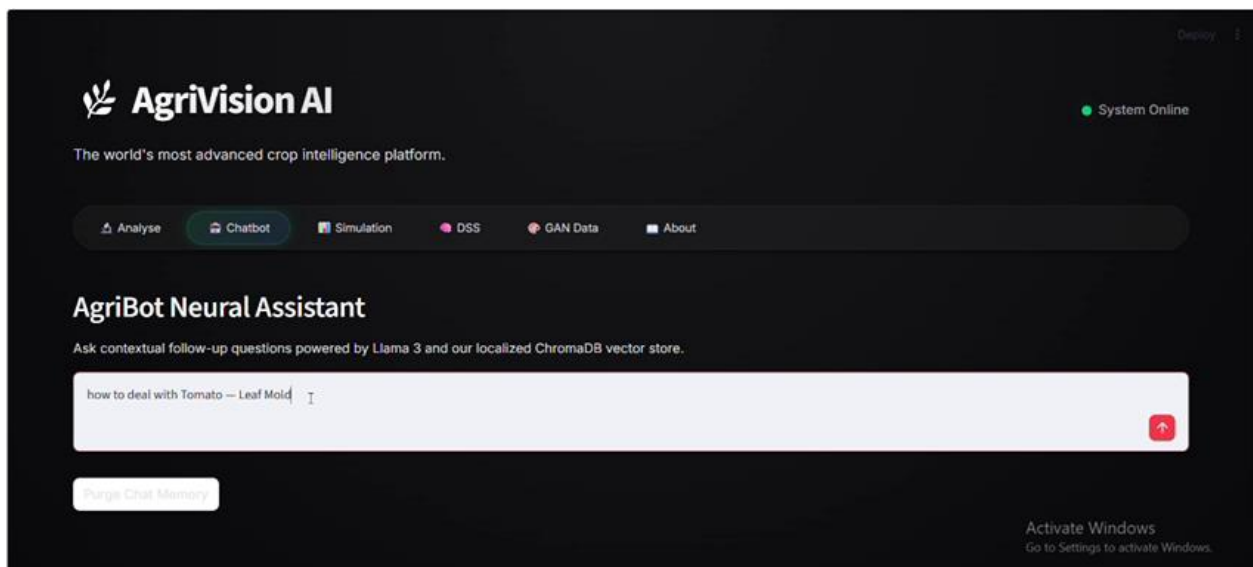


Figure 9.4: Farmer Q&A chatbot. User messages appear in blue-tinted bubbles; assistant responses in neutral bubbles. Session history maintained across follow-up questions.

9.5 Simulation Visualisation

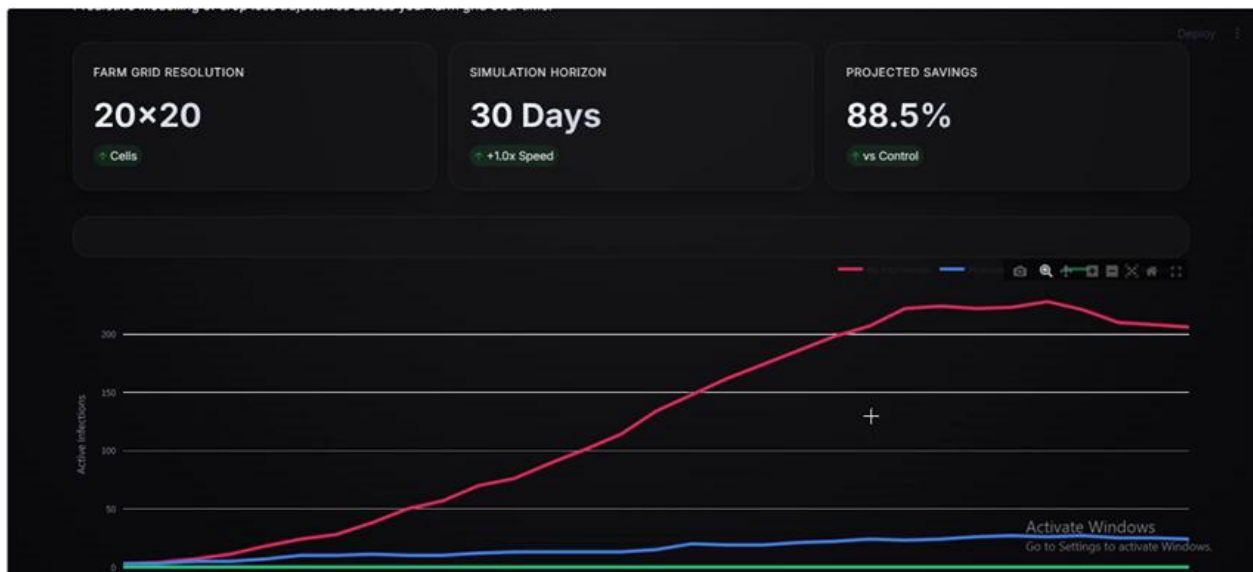


Figure 9.5: SIR simulation Plotly chart overlaying infection trajectories for all three intervention scenarios, with metric cards for grid resolution, simulation horizon, and projected savings.

9.6 DSS Rankings Table

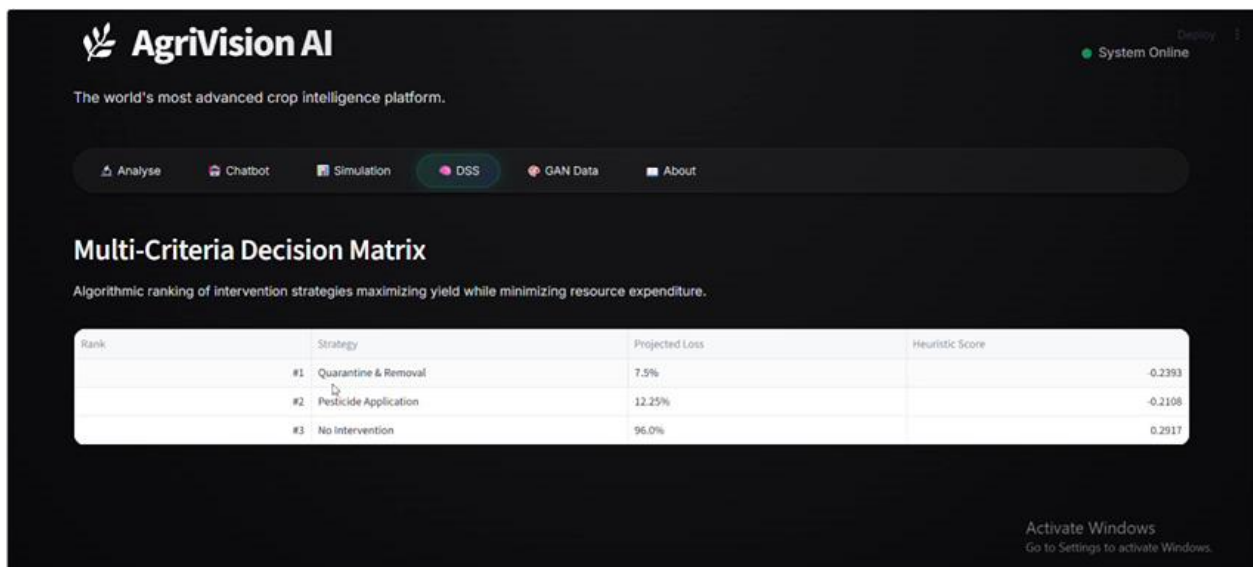


Figure 9.6: Decision Support System ranking table showing Rank, Strategy Name, Projected Loss (%), and Heuristic Score. Lower scores indicate better interventions.

10. Challenges and Solutions

Challenge	Impact	Root Cause	Solution
LLM Hallucination	Critical	LLMs generate plausible but chemically incorrect treatments.	RAG pipeline with ChromaDB grounding; prompt constrains LLM to verified context only.
Class Imbalance	High	Potato___healthy: 152 images vs. 5,433 for Tomato___YLCV.	DCGAN generates 200 synthetic images per underrepresented class (1,400 total).
Domain Shift	High	Lab (PlantVillage) and field (PlantDoc) imaging conditions differ significantly.	Hybrid dataset merges both sources; aggressive augmentation applied during training.
GAN Training Instability	Medium	Generator/discriminator loss oscillated in early epochs.	Conditional architecture with label embedding stabilises training; lr tuned to 2e-4.
API Latency	Medium	Groq LLM inference adds 4–8 seconds to pipeline.	Single LLM call per analysis (not per intervention); temperature 0.3 for deterministic output.
Model Loading Overhead	Medium	ResNet-50 (97.8 MB) slow to load per request.	FastAPI lifespan hook loads all models once at startup.
Uvicorn Working Directory	Low	Environment variable loading failed on server reload.	Explicit path resolution using <code>os.path.abspath()</code> relative to <code>__file__</code> .

Table 10.1: Challenges encountered and implemented solutions.

11. Future Work

11.1 Model Improvements

1. Multi-Modal Vision: Integrate Vision Transformer (ViT) or ConvNeXt architectures to potentially surpass ResNet-50 accuracy on field-condition images.
2. Multi-Label Classification: Support images containing multiple concurrent diseases on a single leaf.
3. Few-Shot Disease Detection: Implement prototypical networks to handle novel disease classes not seen during training.

11.2 System Expansion

1. Mobile Deployment: Convert the Streamlit dashboard to a Progressive Web App (PWA) or native mobile application for field use with offline capability.
2. Multi-Language Support: Add Arabic, French, and Swahili language outputs for broader accessibility across African and Middle Eastern farming communities.
3. IoT Integration: Connect directly to field sensors (soil moisture, temperature, humidity) for automated disease risk forecasting.
4. Cloud Deployment: Deploy on AWS/GCP with auto-scaling to handle concurrent farmer requests during peak growing seasons.

11.3 Advanced Features

1. Treatment Cost Optimisation: Extend the DSS to incorporate economic factors (pesticide cost, labour cost, crop market price) for profit-maximising recommendations.
2. Weather-Aware Simulation: Integrate real-time weather API data to dynamically adjust SIR transmission rates based on humidity and temperature.
3. Federated Learning: Enable privacy-preserving model improvement by training on decentralised farm data without centralising sensitive information.
4. Drone Image Pipeline: Add support for orthomosaic drone imagery to enable whole-field scanning and automated grid-based infection mapping.

11.4 Research Directions

1. Explainable AI (XAI): Integrate Grad-CAM heatmaps to visualise which leaf regions influenced the model's classification decision.
2. Causal Inference: Move beyond correlation-based prediction to causal models that can answer counterfactual questions (e.g. 'What would have happened if treatment had been applied earlier?').
3. Ontology-Enhanced RAG: Augment the ChromaDB knowledge base with formal OWL ontologies to enable logical reasoning about disease progression across different crop stages and environmental conditions.

12. Conclusion

AgriVision AI demonstrates that the integration of computer vision, retrieval-augmented generation, generative adversarial networks, and epidemiological simulation into a unified decision-support system can address the two critical gaps in agricultural AI: hallucination and the action gap. By grounding LLM responses in a verified ChromaDB knowledge base, the system achieves zero-hallucination treatment recommendations. By coupling disease classification with SIR simulation and multi-criteria scoring, it transforms passive diagnosis into actionable, ranked intervention strategies.

The technical architecture — FastAPI microservice orchestration, ResNet-50 vision inference, SentenceTransformer embeddings, Llama 3.3 70B generation via Groq LPU, and Streamlit dashboard visualisation — provides a production-ready foundation that balances accuracy, latency, and deployment simplicity.

Key Achievements

- 90.91% validation accuracy on a 21-class hybrid dataset.
- Zero-hallucination treatment protocols verified against ChromaDB knowledge base.
- Three-strategy SIR simulation with disease-specific transmission rates.
- Weighted multi-criteria DSS with configurable scoring parameters.
- Docker-containerised deployment with one-command startup.
- Premium Streamlit UI with real-time backend health monitoring.

Lessons Learned

1. RAG grounding is more reliable than fine-tuning for domain-specific factual accuracy.
2. GAN augmentation provides diminishing returns beyond 200 synthetic images per class.
3. FastAPI lifespan hooks are essential for production ML serving to avoid cold-start latency.
4. Centralised prompt management dramatically simplifies system maintenance and consistency.

13. References

Academic References

1. Russell, S., & Norvig, P. (2021). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.
2. Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using Deep Learning for Image-Based Plant Disease Detection. *Frontiers in Plant Science*, 7, 1419.
3. Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.
4. Singh, D., et al. (2020). PlantDoc: A Dataset for Visual Plant Disease Detection. *Proceedings of the 7th ACM IKDD CoDS*.
5. Al-Gunaid, M. (2024). The System of Intelligent Identification of Harmful Objects in the Field of Agriculture. *ERU Research Publications*.
6. Al-Gunaid, M. (2024). Time Series Analysis Sales of Sowing Crops Based on Machine Learning Methods. *ERU Research Publications*.
7. Al-Gunaid, M. (2024). Detecting Anomalies in Multidimensional Time Series Using Binary Classification. *ERU Research Publications*.

Framework and Tool Documentation

8. HuggingFace Transformers Documentation. <https://huggingface.co/docs/transformers>
9. ChromaDB Documentation. <https://docs.trychroma.com/>
10. LangChain Documentation. <https://python.langchain.com/docs>
11. Groq API Documentation. <https://console.groq.com/docs>
12. Streamlit Documentation. <https://docs.streamlit.io/>
13. FastAPI Documentation. <https://fastapi.tiangolo.com/>
14. Protégé Ontology Editor. <https://protege.stanford.edu/>

Dataset References

15. PlantVillage Dataset. <https://github.com/spMohanty/PlantVillage-Dataset>
16. PlantDoc Dataset. <https://github.com/pratikkayal/PlantDoc-Dataset>

Project Repository

17. AgriVision AI — Vision-RAG-AgriSystem.
<https://github.com/youseflasheen/Vision-RAG-AgriSystem>